

GTIN Scanner

01 · OVERVIEW

What it replaces, and why it exists

Pickers used to scan barcodes into an Excel macro to check items against contract lines. It worked, but it lived on one machine, had no audit trail, and gave supervisors no visibility into what was happening on the floor in real time.

The GTIN Scanner is a web app pickers open on a handheld device. They scan a badge to start a session, then scan items — the app looks each one up instantly against the current supply contract and flags anything on hold. Supervisors watch every active session from a separate monitor board and can export or close any of them. A second, native client is now in development for the parts of the warehouse where that web app can't reach at all — see "Offline handheld track" below.

184,411

contract lines indexed for
instant lookup

$O(1)$

lookup time per scan —
dictionary, not a search

3 days

session data retention window
(export before then)

02 · ORIGIN STORY

From an Excel macro to a modernization project

This didn't start as a request for a new app — it started as one team's Excel-macro barcode tool that had quietly become load-bearing for receiving. It only runs on one machine, keeps no history once the sheet is closed, and gives nobody outside that machine any visibility into what's been scanned today. It was picked up as a small modernization effort: rebuild the same workflow as a real web app, on the company's actual data platform, without changing what a picker's day looks like.

Two screens, one shared record

The scanning page is one screen, meant to open on an Android handheld — it needs to be sized and laid out for that device. The monitor board and the admin page are separate screens, meant to open on a laptop or a monitor.

PICKER

Handheld

Scans a badge and a location to start a session, then scans items in a loop. Only the device that started a session can end it normally.

SUPERVISOR

Monitor board

A live list of today's sessions — who, where, how many scans, last activity — with drill-in, per-session export, and a Force End for a device that dropped off.

MANAGER

Admin page

Email-allowlisted refresh control for pulling the latest contract data on demand, plus the same Force End control.

This app also needs to keep working in parts of the warehouse with no WiFi. Our plan for that is to use Android — but that's just a plan for now, not something built into the app yet.

Local first, internet second

We want each scan to query our lakehouse directly, so the app can look up a GTIN there the moment it's scanned. Right now we're using mock data instead, because the app doesn't yet have a service principal set up to query the lakehouse on its own. If a scan isn't found in the lakehouse, the app falls back to the GUDID API to fetch the GTIN from there.

1 · SCAN Barcode read, GS1 wrapper parsed, GTIN extracted.	2 · LOCAL LOOKUP Checked against the in-memory contract index. No network call.	3A · HIT Item, on-hold status, and unit-of-measure shown immediately.	3B · MISS Falls back to the FDA's public device-identifier API.
--	---	---	---

If that fallback call also fails — a timeout, a bad connection — the app records it as a clean "Not Found" instead of erroring out, and logs *why* it missed (wrong check digit, not on the current contract, inactive line, etc.) so the miss is explainable later, not just a blank.

EXAMPLE SCAN	ITEM	RESULT
00841098765432	Surgical Mesh 15cm	Found — contract line
00123456789012	Nitrile Gloves, Large	Found — on hold warning
00000000000000	Not on contract	FDA API fallback

Built for the floor, not the demo

Every feature below exists because of something that broke, stalled, or annoyed someone during real use — not a hypothetical.

Hands-free auto-submit

SPEED

PROBLEM

The warehouse's actual scanners (Zebra handhelds) deliver a whole barcode in one instant burst, not as typed keystrokes. The first version of auto-submit was tuned for a keystroke-by-keystroke scanner and simply never fired on the real hardware — pickers had to tap the screen after every item.

SOLUTION

Two independent triggers now cover both delivery styles: scan-speed cadence for guns that type character-by-character, and a check-digit validity test as a backstop for guns that paste the whole code at once. Manual typing still works — Enter or the Look Up button always did.

Audible confirmation

SPEED

PROBLEM

Pickers are watching the item in their hands, not the screen. A quiet or ambiguous tone was easy to miss over warehouse noise, or read as a glitch rather than a confirmation.

SOLUTION

A distinct two-note chime on every successful scan, with separate alarm tones for on-hold and not-found results — loud and clear enough to register without looking up, and unmistakably different from an error.

Session resume after a dropped connection

RELIABILITY

PROBLEM

Warehouse wifi is inconsistent, and mobile browsers routinely kill or reload a backgrounded tab. Losing the tab used to mean losing the session and starting over.

SOLUTION

Every session and scan is written straight to the database, not just held in the browser tab. The session ID lives in the page URL, so a refresh, a killed tab, or a full browser restart reopens the same session with its full scan history intact — nothing is lost, and no scan has to be redone.

Graceful network fallback

RELIABILITY

PROBLEM

The fallback lookup depends on an external, public FDA database over the internet — a connection that isn't always available or fast on a warehouse floor.

SOLUTION

A network failure there never crashes the scan — it resolves to a clean "Not Found" with the reason logged, so the picker keeps moving and the miss is still traceable afterward. And because most scans hit the local index first, this fallback is rarely exercised at all.

Fast cold start

SPEED

PROBLEM

Loading the full 184k-row contract index used to happen before the app showed anything, adding a multi-second blank wait every time the app woke up.

SOLUTION

The index now loads lazily, in the background, after the picker's start screen is already on screen — visible progress instead of a blank wait, with no change to how fast lookups run once scanning starts.

On-hold and duplicate-scan awareness

DATA QUALITY

PROBLEM

A found item isn't automatically a safe item, and rescanning the same barcode twice used to either get silently dropped or inflate the count as if it were a new item.

SOLUTION

Items flagged on hold in the contract data show a distinct warning instead of a plain match. A repeat scan of something already in this session is recognized instantly (no re-lookup) and its count increments, so exports reflect exactly what was physically scanned.

Explainable misses

DATA QUALITY

PROBLEM

A blank "Not Found" gives a supervisor nothing to act on — is the item genuinely new, mislabeled, or just off-contract?

SOLUTION

Every miss is diagnosed and tagged with a specific reason (bad check digit, not on the current contract, inactive line, etc.), stored with the scan and surfaced in both the app and the Excel export.

What we tried first, and why it changed

Four decisions in this app were not the first thing tried. Each is here because the first approach ran into a real constraint — worth knowing before anyone assumes the current design was the obvious starting point.

Contract data source: Redshift → Fabric Lakehouse

Blocked by access, not by engineering

TRIED	Amazon Redshift, the data platform originally assumed to hold the contract-line data.
WHY IT STALLED	Redshift credentials were requested but never issued — the org's actual live data platform is a Microsoft Fabric Lakehouse, not Redshift.
WHAT SHIPPED	The loader was rebuilt against Fabric's SQL analytics endpoint (ODBC, ~184k contract lines), authenticated with Azure AD instead of a stored password. Live since Aug 7, 2026.

Session storage: JSON file → SQLite → hosted Postgres

Two rounds — the first fix wasn't enough either

TRIED	A single JSON file autosaved to local disk, then a local SQLite database, as the record of every session and scan.
WHY IT STALLED	Streamlit Community Cloud restarts the app's container on its own schedule, and its local disk is wiped on every restart — both approaches silently lost every session and scan on the next restart, mid-shift or not.
WHAT SHIPPED	A hosted Postgres database (Neon) that lives outside the app's container entirely, so an app restart is invisible to a picker's in-progress session. A daily scheduled job handles data retention independently of whether the app is even running.

Badge/session linking: in-app camera QR scan

Scoped out before it was built

TRIED

An early design considered scanning a QR code with the device camera, inside the app, to link a handheld to a session.

WHY IT STALLED

Streamlit's UI runs custom components inside an iframe, and iframe-embedded components can't reliably get camera (`getUserMedia`) permission across devices and browsers — this would have spent the riskiest engineering effort in the project on a problem that turned out to have a simpler answer.

WHAT SHIPPED

Sessions are minted directly on the handheld from a typed/scanned badge ID and location — no camera, no QR, no in-app scanning step at all.

Warehouse dead zones: PWA rewrite → native Android client

Decided Sept 23, 2026, after the browser-only ceiling became clear

TRIED

Treating "scanning is online-only" as an accepted v1 limitation, with server-side hardening (an offline write buffer on the app's own connection to Neon) as the mitigation.

WHY IT STALLED

That hardening only protects the server's connection to the database — it cannot help a handheld whose browser has no connection at all. A WiFi dead zone breaks a Streamlit session completely, because Streamlit needs a live round trip for every interaction; no code running on the server can buffer around a browser that can't reach it.

WHAT'S SHIPPING

A native, offline-first Android client for the Zebra HC50 (local SQLite mirror + write queue + DataWedge integration) talking to a new REST service, `sync_api/`, built in this repo. See "Offline handheld track" below for status.

What's stored, and for how long

Sessions and scans are kept for **3 days**, then purged automatically by a daily scheduled job — this runs independently of the app, so it fires even if nobody has the app open. Export any session you need to keep before then; there's no longer-term archive in this version.

Each session can be exported to a single Excel workbook from the monitor board — one file per session, named with the location, picker, and date so a batch of exports stays sortable.

Who can do what

Anyone can pick up a device and start a scanning session — that's the front-line workflow and it's intentionally open. Three surfaces are restricted to a named allowlist of managers and supervisors: the monitor board itself (added in the security hardening pass), refreshing the contract data on demand, and force-ending a session that a device dropped without properly closing.

Note on the access model: admin and board access today is a typed-email allowlist check, not a verified login — closer to a sign-in sheet than a badge reader. Every attempt, granted or denied, is logged, so misuse is visible after the fact even though it isn't blocked up front. This is a known, deliberate trade-off, accepted pending a verified identity provider — see "Security posture" below.

A native client for warehouse dead zones

The browser app requires a live round trip for every scan, so a WiFi dead zone breaks it entirely — no amount of server-side work can fix that, because the server never hears from a handheld that can't reach it. Decided Sept 23, 2026: a native, offline-first Android client for the Zebra HC50 handhelds, backed by a new REST service (`sync_api/`) built in this repo. The handheld keeps a local mirror of the contract catalog and a queue of pending operations on-device; it resolves scans against that mirror immediately and uploads the queue whenever it reconnects. A mirror miss shows as "not on contract file — will be verified when online," never a final Not Found, since the authoritative record — including the FDA fallback lookup, which can't be mirrored — is created server-side at sync time.

COMPONENT	STATUS
<code>sync_api/</code> — REST sync service	Built and tested; documented in SYNC-API.md. Needs HTTPS termination and must run as a single process (its rate limiter and reference snapshot are in-process state).
<code>android/core</code> — scan & sync logic (pure Kotlin/JVM)	Built and tested: 37 unit tests plus a live end-to-end test against the real service.
<code>android/app</code> — Android layer (SQLite, DataWedge, WorkManager, UI)	Written and type-checked against the Android 14 framework; the Gradle build has never actually been run. Current UI is a functional stub.
MDM distribution	Not started. Release builds take the sync URL and token from MDM-pushed Managed Configuration; HTTPS is mandatory there.
Field validation on real HC50 hardware	Not started — the DataWedge integration follows Zebra's documented SET_CONFIG API but is unverified on a device.

Full wire contract, error semantics, and security properties for the sync service are in SYNC-API.md; client build and status detail is in android/README.md.

Audited and hardened; two items need infrastructure, not code

A full ASVS (OWASP Application Security Verification Standard) review and remediation pass completed Sept 14, 2026, covering: an application-level egress allowlist; circuit breakers and bounded retry on every external call; GTIN input validation before use; Excel/CSV formula-injection neutralization; encryption at rest (OS-keychain-backed) for the offline write queue, the contract-data cache, and the admin audit log; TLS enforcement, regression-tested; log sanitization; and a pinned dependency lockfile with zero known vulnerabilities. 96 automated tests cover these changes. Full detail is in the companion Production Security Briefing and in `ASVS-COMPLIANCE.md` in the repository.

Everything fixable from inside the application is fixed. What remains needs an action outside the codebase:

ITEM	STATUS
Verified admin / monitor-board identity	Open
Network-level egress restriction at the hosting layer	Open
Production secrets vault	Interim (OS keychain)
MDM / device-lock policy for handhelds	Open
<code>sync_api</code> TLS termination point	Open

Detail on each is in "Outstanding requirements" below, alongside the hosting and offline-client items.

Current hosting and technical footprint

The app and its data connections are real, live, and in production use. It currently runs on **Streamlit Community Cloud**, a free public hosting tier meant for demos and personal projects: the app's source deploys straight from GitHub, sleeps after inactivity, and runs on shared infrastructure outside the organization's own systems. A move to **Sanford's own internal infrastructure** was decided on Sept 23, 2026, specifically so that network-level controls (NSG/firewall/private-endpoint rules) can be enforced directly — Streamlit Community Cloud exposes none of that. The move itself has not yet happened.

Every scan and session a picker records — who scanned what, when, and from where — is written straight to a **Neon Postgres database**, a hosted cloud database separate from both Streamlit and the Fabric catalog. That's what makes a session survive a dropped connection or a restarted app (see "Session resume after a dropped connection" above): the record lives outside the app itself, not in the picker's browser.

APPLICATION

Framework
Python 3.11+ / Streamlit

Current host
Streamlit Community Cloud

Target host
Sanford internal infra (decided; not yet provisioned)

Source control
GitHub (private repo)

DATA CONNECTIONS

Product catalog (contract lines)
Microsoft Fabric Lakehouse, via ODBC

Catalog sign-in
Azure AD (interactive today)

Scan & session records
Neon Postgres (hosted database)

Fallback lookup
Public FDA device database (no login needed)

BACKGROUND JOBS

Old data cleanup
Daily, GitHub Actions

Catalog refresh
Manual button + 24h auto-refresh

SECRETS

Local dev
OS keychain, .env fallback

Hosted today
Streamlit Cloud secrets store

Needed for production
Org secret manager / vault

Infrastructure requirements

- 1 Internal hosting.** Decided Sept 23, 2026. The app and the new `sync_api` service both need a host inside Sanford's own infrastructure, with outbound network access to the Fabric SQL endpoint and to Neon (or an internal Postgres instance) — not yet provisioned.
- 2 Unattended sign-in to Fabric.** A service-principal app registration in Azure AD, scoped read-only to the contract-line table, is supported in the app already (`FABRIC_AUTH=serviceprincipal`) — the credential itself has not yet been issued.
- 3 Managed secrets.** The service-principal secret, the database connection string, and the admin allowlist need a production secret store in place of the app's own hosting config.
- 4 Session/scan storage location.** Neon is already working well; an internal Postgres instance is the alternative if data residency ever requires storage to stay inside the network.
- 5 Verified admin access.** Today's typed-email allowlist needs to be replaced by real sign-in against the company's Microsoft identity; see "Access & audit" above.

What production readiness still needs

Consolidated from "Security posture" and "Where it runs today" above — everything still required before this is a fully production-owned, internally hosted system with an offline-capable handheld fleet.

- **Internal hosting.** Provision a host inside Sanford's infrastructure for the app and `sync_api`. Decided; not yet done.
- **Fabric service-principal credential.** Required for unattended production sign-in to the Lakehouse. Not yet issued.
- **Verified admin / monitor-board identity.** An OIDC app registration (Microsoft Entra fits the existing company email domains) to replace the typed-email allowlist. Not yet provisioned.
- **Production secrets vault.** A managed secret store for the service-principal secret and the database connection string. Not yet selected.
- **Network egress restriction.** A firewall/NSG rule limiting the app's outbound traffic to Fabric, Neon, and the FDA AccessGUDID API — enforced in application code today; the network-level rule depends on where hosting lands.
- **MDM / device policy.** A Managed-Configuration-capable MDM platform, plus a lock-screen/auto-lock policy, for the Zebra HC50 handhelds. Not yet selected.
- **`sync_api` TLS termination.** Must be served over HTTPS in production. Termination point (reverse proxy vs. certs on the service) not yet decided.
- **Android app build & field validation.** `android/core` is tested; `android/app` has never actually been built or run. Needs an Android Studio build pass and testing on real HC50 hardware.
- **Android release readiness.** A real UI, provisioning error screens, an instrumented test suite, release signing, and MDM distribution all remain before a field pilot.

GTIN Scanner — internal supply-chain tooling. This document reflects the app as of Oct 5, 2026.